COMPTIA PENTEST+ · **PT0-003** · DOMAIN 4.9

Attacks Against Specialized Systems

Specialized systems — mobile, AI, operational technology, and short-range wireless — were built for function, automation, or portability, not security. This domain is **explain**-level and heavily scenario-based: given a situation, name the attack, the vulnerability, and the right tool. This guide weaves the exam tells directly into each concept.

6 attack families · + toolkit | Objective verb: Explain | Format: scenario-based

1 · Mobile | 2 · AI | 3 · OT | 4 · NFC | 5 · Bluejacking | 6 · RFID | 7 · BT Spam | 8 · Tools | Correlations | Strategy

SEC 01 Mobile Attacks

Phones store credentials, tokens, and PII, connect to corporate networks, and go everywhere. Mobile attacks cluster around three outcomes: **extracting data**, **escalating privilege** (root/jailbreak), and **abusing the permission model**.

1.1 Information Disclosure

The unauthorized exposure of data stored on or transmitted by a device. Apps routinely stash data locally in SQLite databases, shared-preferences files, logs, caches, and temp directories. When those aren't secured, a tester can pull usernames, passwords, session tokens, API keys, PII, and financial data.

Disclosure travels several paths: **local storage exposure** (plaintext data in `/data/data/<package>/`, reachable on rooted devices or via ADB); **insecure transmission** (no TLS or weak ciphers, interceptable on-path); **log leakage** (debug statements writing secrets to logcat); **clipboard/screenshot** capture; and **backup vulnerabilities**.

⚑ EXAM TELL

If a scenario says `allowBackup=true` in `AndroidManifest.xml`, that's the flag: app data can be extracted **without root**. And logs readable via `adb logcat` = runtime data disclosure.

Finding disclosure needs both **static analysis** (reading code and config) and **dynamic analysis** (watching runtime behavior). Scenarios that say "reviewed the code" point at static; "observed the running app" points at dynamic.

EXAM FOCUS

- Plaintext SQLite / shared prefs / log files = insecure storage findings.
- `allowBackup=true` → data extraction without root.
- Static = code review; Dynamic = runtime observation.

plaintext storage | insecure logging | logcat | allowBackup | cleartext credentials | ADB extraction

1.2 Jailbreak / Rooting

Removing the manufacturer's restrictions to gain administrative (root) access. **Rooting = Android**; **Jailbreaking = iOS** — memorize the pairing. Android rooting exploits bootloader/kernel/OS flaws to unlock Linux root; iOS jailbreaking exploits kernel or bootrom weaknesses to defeat code signing, sandboxing, and Secure Boot.

⚠ WATCH THE TRAP

Treat a rooted/jailbroken device as **fully compromised by default**. Once root is achieved: sandboxes fall, Keychain (iOS) / Keystore (Android) secrets are extractable, certificate pinning can be bypassed, and all local data is readable regardless of file permissions.

Testers often receive a rooted/jailbroken device in scope so they can go deeper. Many apps try to **detect** root/jailbreak and refuse to run — testers hook those checks with **Frida** or **Objection** to return false negatives (detection evasion).

EXAM FOCUS

- Rooting = Android, Jailbreaking = iOS.
- Bypasses sandboxing, code signing, Secure Boot.
- Frida / Objection defeat root/jailbreak detection.

`root access` `sandbox bypass` `code signing bypass` `Secure Boot` `Keychain` `Keystore` `SELinux bypass` `Frida bypass` `detection evasion`

1.3 Permission Abuse

Apps declare the permissions they need; abuse happens when an app is **over-privileged** (asks for camera, mic, contacts, SMS to show static content) or when Android components are misconfigured. Over-privilege matters because each granted permission widens the attack surface if the app is compromised.

The component-level issues: **exported components** (`exported=true` on Activities, Services, Receivers, Content Providers) are callable by any other app with no permission; **intent hijacking** lets a malicious app intercept implicit Intents; and **content provider** misconfigurations let any app query, insert, update, or delete data.

⚑ EXAM TELL

Any scenario about `exported=true`, implicit intents, or content providers is **permission abuse**, and the tool is **Drozer** — it talks to apps through Android IPC exactly the way another app would.

EXAM FOCUS

- Exported components (`exported=true`) = accessible to other apps, no permission needed.
- Implicit intents can be hijacked; content providers can leak data.
- Drozer is the tool for exported components + content providers.

`exported components` `AndroidManifest.xml` `over-privileged` `implicit intent` `content provider` `permission escalation` `Drozer`

SEC 02 AI Attacks

AI systems interpret language and make probabilistic decisions, creating a surface that doesn't exist in deterministic software. Two attack types matter here: **prompt injection** (runtime, through the interface) and **model manipulation** (structural, through training or deployment).

2.1 Prompt Injection

Malicious input that makes an LLM ignore its instructions, reveal secrets, or take unintended actions. It's the AI analogue of SQL injection: user-supplied text gets treated as instructions rather than data. The model processes system prompt and user input in one context window and can't inherently tell trusted instructions from injected ones.

Two forms: **direct** (attacker types the malicious instructions straight into the interface — "ignore all previous instructions...") and **indirect** (instructions hidden in content the AI will later read — a web page, document, or email). Indirect is especially dangerous for autonomous agents that browse or read files.

> ⚑ **EXAM TELL**
>
> "Manipulating the AI through input/prompts" → **prompt injection**. If the AI agent has **tool access** (web browsing, file reading, API calls), treat it as a high-value target — that's the detail that raises severity in a scenario.

EXAM FOCUS

- Prompt injection = input overriding AI system instructions.
- Analogous to SQL injection; targets AI reasoning, not queries.
- Direct (typed in) vs. indirect (embedded in processed content).

`system prompt override` `LLM` `instruction injection` `content filter bypass` `indirect injection` `AI agent`

2.2 Model Manipulation

Attacks on the model itself — training data, parameters, weights, or deployment environment — rather than through its interface. The compromise is embedded in the model, making it subtle, durable, and hard to detect.

- **Data poisoning:** corrupt training data so the model learns wrong associations (e.g., a spam filter trained to pass malicious mail).
- **Backdoor / Trojan models:** a hidden trigger causes attacker-controlled output only on a specific input; benchmarks look normal.
- **Model inversion:** reconstruct training data (potentially PII) by querying and analyzing outputs.
- **Adversarial examples:** human-imperceptible input tweaks that force misclassification — used to bypass AI malware detection, facial recognition, or IDS.
- **Model theft / extraction:** query heavily to train a surrogate that approximates the original (IP theft + offline attack platform).
- **Deployment attacks:** compromise the serving platform, container, or API gateway to alter weights or swap the model.

> ⇄ **DISTINGUISH**
>
> Corrupt training data, embed a backdoor, or compromise the deployment environment → **model manipulation**. Manipulate through prompts/input → **prompt injection**. This is the exact fork the exam tests.

EXAM FOCUS

- Prompt injection = runtime input; model manipulation = structural/training-level.
- Poisoning corrupts training; backdoors add hidden triggers.
- Adversarial examples defeat AI-based detection; extraction steals IP.

`data poisoning` `backdoor trigger` `adversarial example` `model inversion` `training data` `surrogate model` `model extraction` `deployment compromise`

SEC 03 Operational Technology (OT)

OT — ICS, SCADA, PLCs, DCS — controls physical processes: power grids, water treatment, manufacturing, pipelines. OT attacks are uniquely dangerous because they cause **physical** harm. Testing favors **passive assessment**; aggressive scanning can trigger real operational failures.

3.1 Register Manipulation

PLCs and RTUs store operational data in memory registers — the interface between the digital controller and the physical world. In Modbus terms: **coils** (1-bit read/write outputs — valves, motors), **discrete inputs** (1-bit read-only sensors), **holding registers** (16-bit read/write setpoints), **input registers** (16-bit read-only measurements).

With network access, an attacker sends unauthorized Modbus commands to read or write these values. Writing a false temperature setpoint can overheat a furnace; writing a coil can open or close a valve unexpectedly.

⚠ WATCH THE TRAP

Register manipulation has **physical** consequences — that's what separates it from IT attacks. Because writes can be immediate and irreversible, OT testing usually assesses this **passively** rather than by active exploitation.

EXAM FOCUS

- Registers control physical outputs (valves, motors, setpoints).
- Modbus is the protocol used to read/write registers.
- Physical harm distinguishes it from IT; passive assessment preferred.

PLC · RTU · holding register · coil · Modbus · setpoint manipulation · ICS

3.2 CAN Bus Attack

The Controller Area Network is the backbone bus inside virtually every modern vehicle, linking the ECU, transmission, ABS, airbags, and infotainment — and it's used in industrial machinery, robotics, and medical devices too. It's a two-wire differential serial bus where messages carry an **arbitration ID** (not an address), every node hears every message, and there is **no authentication or encryption**.

Attack vectors: **physical access** via the OBD-II port (inject arbitrary frames); **remote** via telematics/Bluetooth/Wi-Fi (the famous Jeep Cherokee hack); **message injection** spoofing any ECU; **fuzzing** to crash ECUs; and **DoS** by flooding low-arbitration-ID (high-priority) messages.

⚑ EXAM TELL

"Vehicle" or "ECU" in a scenario → CAN bus. The **OBD-II port** is the direct physical entry point; "no authentication" is why message injection works. Remember lower arbitration ID = higher priority (relevant to the DoS flood).

EXAM FOCUS

- CAN bus = vehicle/industrial network, no authentication.
- OBD-II = direct physical access; telematics/BT/Wi-Fi = remote.
- Injection spoofs any ECU; fuzzing causes safety failures.

CAN bus · ECU · OBD-II · message injection · CAN spoofing · telematics · arbitration ID · vehicle hacking

3.3 Modbus Attack

Modbus (1979, Modicon) is still dominant in SCADA/ICS thanks to its simplicity and huge legacy install base. It runs a primary/secondary (master/slave) model and has **no authentication, no encryption, no authorization** — any device that can reach a slave can read or write any register.

- **Unauthorized reads:** FC03 (holding registers), FC01 (coils) reveal live process values.
- **Unauthorized writes:** FC06 (single register), FC16 (multiple), FC05 (single coil) — the dangerous capability.
- **Recon:** FC43 (Read Device Identification) enumerates vendor, product, firmware.
- **DoS:** flood the slave so legitimate master commands can't process.

⚑ EXAM TELL

Modbus/TCP = port 502. Memorize the function codes — FC03 read holding, FC06 write single, FC01 read coils, FC05 write single coil. Modbus attacks need only network access; **no credentials**.

EXAM FOCUS

- No auth / no encryption / no authorization.
- TCP port 502 for Modbus/TCP.
- Scapy and Wireshark are the go-to tools.

`Modbus` `TCP port 502` `function code` `holding register` `coil` `no authentication` `SCADA` `primary/secondary`

3.4 Plaintext Attack

In OT, this is the exploitation of industrial protocols that transmit everything — commands, setpoints, credentials, data — in cleartext. Affected: **Modbus, DNP3, PROFINET, BACnet, OPC (legacy), Telnet, FTP**.

Mechanics: **passive sniffing** (Wireshark/tcpdump reads all traffic undetected), **credential harvesting** (Telnet/HTTP management interfaces), **command replay**, and **traffic analysis** that reveals operating schedules and normal parameters.

⇄ DISTINGUISH

OT plaintext ≠ IT plaintext: remediation is far harder. Legacy devices can't be updated, encryption adds latency that breaks real-time requirements, and replacing equipment is expensive — that's **why it persists**, and the exam likes that reasoning.

EXAM FOCUS

- Most OT protocols are plaintext by design (Modbus, DNP3, BACnet, PROFINET).
- Passive sniffing captures everything without detection.
- Plaintext capture enables command replay.

`cleartext` `no encryption` `passive sniffing` `DNP3` `BACnet` `Telnet` `credential harvesting` `traffic analysis`

3.5 Replay Attack (OT context)

Capture legitimate control commands off the wire and retransmit them later. OT protocols lack timestamps, sequence numbers, nonces, and MACs, so a PLC can't tell a replayed command from the real one. Flow: **capture** (Wireshark/tcpdump) → **analyze** → **replay** (tcpreplay or Scapy).

⇄ DISTINGUISH

IT replay usually targets **authentication** (Kerberos, NTLM) and is mitigated by timestamps/nonces. OT replay targets **operational commands** with physical results — replaying a shutdown halts a line; replaying a valve-open causes overflow. **tcpreplay** replays a captured PCAP; **Scapy** crafts modified replays.

EXAM FOCUS

- No replay protection in Modbus, DNP3, most OT protocols.
- tcpreplay = replay captured PCAP; Scapy = craft modified packets.
- OT replay = physical consequences vs. IT replay = auth bypass.

captured commands · PCAP replay · tcpreplay · Scapy · no sequence numbers · no nonce · Modbus replay

SEC 04 Near-Field Communication (NFC)

Short-range wireless (~4 cm) at **13.56 MHz**, used in contactless payments, access badges, and transit cards. Its tiny range was meant as a security feature — proximity implies consent — but several attacks break that assumption.

4· NFC Attacks

Three modes: **card emulation** (device acts as a card — mobile pay), **reader/writer** (reads/writes tags), and **peer-to-peer**. The attacks:

- **Eavesdropping:** directional antennas + SDR capture the RF beyond the intended 4 cm.
- **Relay attacks:** two devices — a "mole" near the reader and a "ghost" near the victim's card — proxy the transaction across a much greater distance; the reader thinks the card is right there.
- **Cloning:** tags/cards without crypto challenge-response have their UID and data copied to a blank tag.
- **Data manipulation:** in reader/writer mode, overwrite a writable tag (e.g., a poster tag redirecting to a malicious URL).
- **Amplification:** high-gain antennas + SDR extend attack range past 4 cm.

⚑ EXAM TELL

"Access badge" + relay = the classic NFC scenario: an attacker near an employee relays to an accomplice at the reader. Anchor the two numbers — **13.56 MHz, ~4 cm** — they're how the exam separates NFC from LF RFID (125 kHz).

EXAM FOCUS

- NFC = 13.56 MHz, ~4 cm range.
- Relay extends range with two proxying devices; SDR extends eavesdropping.
- Non-crypto tags can be cloned; badges are relay/clone targets.

NFC · 13.56 MHz · relay attack · eavesdropping · amplification · SDR · NFC cloning · contactless payment · proximity bypass

SEC 05

Bluejacking

The most basic Bluetooth attack — and a useful entry point to the wider Bluetooth surface. Bluejacking = sending **unsolicited messages** to discoverable devices. Despite the name, it does **not** hijack anything.

5·

Bluejacking

It abuses the Bluetooth OBEX Push profile (built for simple exchanges like sharing a vCard). Flow: scan for discoverable devices in range (10–100 m for Class 1) → send an unsolicited OBEX Push message/card/file → the target sees an accept/decline prompt. Even a decline means the notification was delivered.

Low impact alone, but it's a **social engineering vector** ("you've won a prize → [URL]"), a **malware delivery** path if a pushed file is accepted and opened, and a **recon** step exposing BDAddr, device names, and classes. It's a precursor to Bluesnarfing and Bluebugging.

⇄ DISTINGUISH

Bluejacking sends a message — it does **not** steal data or grant access. That's **Bluesnarfing** (data theft) and **Bluebugging** (command execution). And every Bluetooth proximity attack requires the target in **discoverable mode**.

EXAM FOCUS

- Unsolicited messages via OBEX Push; needs discoverable mode.
- Social engineering + malware delivery are the offensive uses.
- ≠ Bluesnarfing (theft) and ≠ Bluebugging (command execution).

Bluetooth discoverable mode · OBEX Push · unsolicited message · social engineering · BDAddr · Bluesnarfing · Bluebugging

SEC 06

Radio-Frequency Identification (RFID)

RFID uses electromagnetic fields to identify tags — badges, supply chain, passports, payment cards. NFC is a 13.56 MHz subset of RFID; RFID spans more frequencies. Core relevance: **physical access control bypass**.

6· RFID Attacks

Frequency bands drive the risk: **LF 125 kHz** (HID Prox, EM4100 — ~10 cm, no encryption, easily cloned), **HF 13.56 MHz** (MIFARE, iCLASS — some encryption; this is NFC's band), and **UHF 860–960 MHz** (supply chain, up to ~12 m).

- **Cloning:** the most common attack — read a tag (e.g., with a Proxmark), copy the UID to a blank writable card. LF cards store just a fixed UID, so this is trivial.
- **Eavesdropping:** passive sniffing of card↔reader traffic; encrypted exchanges captured for offline analysis.
- **Relay:** two devices extend range beyond the intended distance.
- **Replay:** static identifiers with no challenge-response can be replayed for access.
- **Jamming:** RF noise creates a DoS; **brute force** works on short key spaces.

⚑ EXAM TELL

"Access badge" or "clone a badge" → RFID, and the tool is the **Proxmark3**. Nail the LF/HF split: **125 kHz = no encryption, easily cloned** (HID Prox/EM4100) vs. **13.56 MHz = NFC band, some crypto** (MIFARE/iCLASS).

EXAM FOCUS

- LF 125 kHz = no encryption, easily cloned; HF 13.56 MHz = some encryption.
- Proxmark3 is the primary RFID cloning/analysis tool.
- Cloning enables physical access control bypass.

`RFID cloning` `Proxmark` `125 kHz` `HID Prox` `EM4100` `MIFARE` `access badge` `relay attack` `no encryption`

SEC 07 Bluetooth Spamming

The high-volume evolution of Bluejacking: instead of one message, spamming repeatedly floods pairing requests, advertising packets, or file transfers to overwhelm a target or wear down the user.

7· Bluetooth Spamming

Goals: DoS the Bluetooth stack, confuse users into accepting requests, deliver payloads via accepted transfers, or impersonate legitimate devices. **Modern BLE advertising spam** uses tools like the Flipper Zero to broadcast spoofed BLE ads impersonating Apple devices, spamming nearby iPhones with pairing notifications — a UI-level DoS and a social-engineering lever.

- **Connection flooding:** continuous L2CAP/RFCOMM attempts can crash vulnerable stacks.
- **File transfer spam:** repeated OBEX Push with malicious files.
- **Advertising / pairing spam:** repeated prompts cause user fatigue → accidental acceptance.

⇄ DISTINGUISH

Bluejacking = one message; **Bluetooth spamming = high volume**, with potential DoS. The exam-listed active tool is **Bluestrike** — the choice when a scenario calls for active Bluetooth exploitation rather than passive monitoring.

EXAM FOCUS

- High-volume repeated messages/requests; can DoS weak stacks.
- BLE advertising spam exploits proximity notification frameworks.
- Bluestrike is the listed tool; malware delivery via accepted files is the main harm.

`BLE advertising` `OBEX flooding` `pairing request spam` `Bluestrike` `Flipper Zero` `BLE DoS` `user fatigue`

SEC 08 The Domain 4.9 Toolkit

The exam tests **tool ↔ scenario** matching. Be able to state each tool's purpose in one sentence. Note the clean split: OT tools (Scapy, tcpreplay, Wireshark/tcpdump) vs. mobile tools (MobSF, Frida, Drozer, ADB) vs. Bluetooth (Bluestrike).

8.1 Scapy

Python packet-manipulation library — forge, decode, send, and capture packets across hundreds of protocols at Layer 2+. In 4.9 it's the OT **crafting** tool: custom Modbus/TCP packets, CAN frame injection, protocol fuzzing, and modified replay construction. **Scapy builds packets from scratch** — contrast with tcpreplay, which replays real captures.

`custom packet crafting` `Modbus testing` `CAN injection` `OT fuzzing` `Python-based`

8.2 tcpreplay

Suite for editing and replaying captured PCAP traffic — the primary OT **replay** tool. `tcpreplay` retransmits captures (original/custom rate), `tcprewrite` edits headers (IP/MAC/port) before replay, `tcpprep` preprocesses. Use it when the legitimate command format is too complex to rebuild by hand.

`PCAP replay` `OT replay attack` `tcprewrite` `timing control`

8.3 Wireshark / tcpdump

The capture/analysis pair — the OT recon tools. **Wireshark** (GUI) dissects Modbus/DNP3/BACnet/PROFINET/CAN into human-readable fields and saves PCAPs. **tcpdump** (CLI) is lightweight for remote/constrained hosts. In OT, **passive monitoring via port mirroring** is preferred — capture without disrupting operations.

`passive capture` `PCAP` `Modbus dissection` `port mirroring` `no disruption`

8.4 MobSF (Mobile Security Framework)

Automated mobile app testing — both static and dynamic. **Static:** decompiles APK/IPA, reads `AndroidManifest.xml` for dangerous permissions / exported components / backup settings, finds hardcoded secrets. **Dynamic:** runs the app, monitors traffic, hooks calls. **The go-to for discovering information disclosure and permission abuse.**

`static analysis` `dynamic analysis` `APK/IPA` `hardcoded secrets` `exported components`

8.5 Frida

Dynamic instrumentation — inject JavaScript into running processes to hook and modify behavior at runtime. Uses: **bypass SSL/TLS pinning**, **bypass root/jailbreak detection**, extract encryption keys, observe data pre-encryption. **Objection** automates common Frida tasks. Needs a rooted/jailbroken device or a repackaged app — which ties it back to the root/jailbreak concept.

`runtime instrumentation` `function hooking` `SSL pinning bypass` `root detection bypass` `Objection`

8.6 Drozer

Android security assessment framework that talks to apps through **Android IPC** — like another app would. Enumerates and interacts with exported components, exploits content providers (query/insert/update/delete), injects intents, and maps attack surface. **The primary tool for permission-abuse testing. Android-only** — for iOS runtime work use Frida/Objection.

`Android IPC` `exported components` `content provider` `intent injection` `Android-only`

8.7 Android Debug Bridge (ADB)

Command-line interface to Android devices. `adb shell` (root shell on rooted devices), `adb push/pull` (transfer data/tools), `adb install`, `adb logcat` (system logs), port forwarding, and `adb backup` (extracts data when `allowBackup=true`). **The primary data-extraction mechanism — the hands-on tool behind information disclosure.** Network ADB without auth is a critical misconfiguration.

`adb shell` `adb logcat` `adb pull` `adb backup` `allowBackup` `network ADB`

8.8 Bluestrike

The PT0-003-listed Bluetooth exploitation tool: scan discoverable/BLE devices, enumerate (BDAddr, name, class, profiles), and execute active attacks including spamming and protocol exploitation. **Select Bluestrike when a scenario needs active Bluetooth exploitation** — where Wireshark passively captures, Bluestrike actively attacks.

`Bluetooth exploitation` `BLE attacks` `active Bluetooth attack` `spamming tool` `Bluestrike`

MAP Master Correlation Guide

Scenarios chain concepts together. These are the five chains the exam builds questions from.

1 · Mobile Attack Chain

`Jailbreak/Root` → `Frida (bypass detection)` → `ADB (extract data)` → `MobSF (analyze)` → `Drozer (test components)` → `Info Disclosure`

Rooting enables Frida; Frida bypasses SSL pinning; ADB extracts logs; MobSF does static analysis; Drozer tests exported components. **All paths end at information disclosure.**

2 · OT Attack Chain

`Passive Capture (Wireshark/tcpdump)` → `Plaintext read` → `Identify Modbus` → `Register Manip (Scapy)` → `Replay (tcpreplay)` → `Physical Effect`

OT always starts **passive** (no disruption), then understands the protocol, then escalates to active manipulation. Tools map directly: Wireshark capture, Scapy craft, tcpreplay replay.

3 · Wireless Attack Chain

`RFID/NFC discovery` → `Clone / Relay` → `Physical Access` → `BT Scan (Bluestrike)` → `Bluejack / Spam` → `Social Engineering`

Physical pentests combine **RFID badge cloning for entry** with **Bluetooth social engineering once inside**. NFC relay bypasses proximity; BT spam creates confusion.

4 · AI Attack Distinction

`Through input/prompts` → `Prompt Injection` `Training data / weights / deploy env` → `Model Manipulation`

Runtime interface attack = **prompt injection**. Corrupt training, embed backdoors, or compromise deployment = **model manipulation**.

5 · Tool Selection Matrix

SCENARIO	TOOL
Craft custom Modbus packets	Scapy
Replay captured OT traffic	tcpreplay
Capture OT traffic passively	Wireshark / tcpdump
Analyze an Android APK statically	MobSF
Bypass SSL pinning at runtime	Frida
Test exported Android components	Drozer
Extract data from an Android device	ADB
Execute a Bluetooth spamming attack	Bluestrike
Clone an RFID access badge	Proxmark3

KEYS Critical Keyword Master List

Mobile

jailbreak · rooting · sandbox bypass · SSL pinning bypass · allowBackup · AndroidManifest.xml · exported components · content provider · implicit intent · logcat · adb pull · MobSF · Frida · Drozer · Objection · over-privileged

AI

prompt injection · system prompt override · LLM · data poisoning · backdoor trigger · adversarial example · model inversion · model extraction · indirect prompt injection · AI agent

OT

PLC · RTU · SCADA · ICS · Modbus · TCP port 502 · function code · holding register · coil · register manipulation · CAN bus · ECU · OBD-II · DNP3 · BACnet · tcpreplay · Scapy · passive assessment · port mirroring

NFC / RFID / Bluetooth

NFC · 13.56 MHz · relay attack · RFID cloning · Proxmark · 125 kHz · HID Prox · EM4100 · MIFARE · access badge · discoverable mode · OBEX Push · Bluejacking · Bluetooth spamming · BLE advertising · Bluestrike · SDR

Tools (one-line purpose)

- `Scapy = craft packets`
- `tcpreplay = replay PCAP`
- `Wireshark = GUI capture`
- `tcpdump = CLI capture`
- `MobSF = static+dynamic mobile`
- `Frida = runtime hook`
- `Drozer = Android IPC`
- `ADB = Android shell`
- `Bluestrike = Bluetooth exploit`

EXAM Final Strategy for Domain 4.9

01 **Match attacks to systems.** Mobile → MobSF / Frida / Drozer / ADB. OT → Scapy / tcpreplay / Wireshark. Bluetooth → Bluestrike. RFID → Proxmark3.

02 **Understand consequences.** OT = physical harm. Mobile = data exposure. AI = integrity/availability. The consequence type often reveals the domain.

03 **Passive before active.** OT testing favors passive assessment — Wireshark/tcpdump first; Scapy/tcpreplay is the escalation step.

04 **Distinguish similar concepts.** Bluejacking (one message) ≠ Bluetooth spamming (high volume). Prompt injection (runtime input) ≠ model manipulation (structural). Jailbreak (iOS) ≠ rooting (Android). NFC (13.56 MHz, ~4 cm) ≠ LF RFID (125 kHz, ~10 cm).

05 **One-sentence tool purpose.** Be able to state what each tool does in a single line — questions frequently ask "which tool accomplishes X?"

06 **Read for keywords.** "industrial control system" → OT; "Android application" → Drozer/ADB/MobSF; "access badge" → RFID cloning; "AI chatbot" → prompt injection; "Bluetooth device" → Bluejacking/Bluestrike.

Study aid — condensed for review. Verify tool syntax and current exam objectives against official CompTIA materials.